

PART I**C++ FUNDAMENTALS**

From "what is programming?" to functions and pointers

This Part assumes zero prior programming knowledge. If you already know some C++, don't skip it — read it for speed and precision, not novelty. Every idea here is something you will use hundreds of times in every contest and interview you ever do.

Contents of Part I

- 1. What Is Programming?
- 2. Your First C++ Program
- 3. Variables, Constants, and Data Types
- 4. Input and Output for Competitive Programming
- 5. Operators
- 6. Conditions
- 7. Loops
- 8. Functions
- 9. Pointers and References
- Part I Revision, Cheat Sheet, and Interview Questions

Chapter 1 — What Is Programming?

1.1 What is a program?

In one sentence: a program is a list of exact instructions that a computer follows, in order, to do something.

A computer cannot guess what you mean. If you tell a human "make me tea," they fill in a hundred unstated details from experience. A computer has no experience to fill gaps with — it will do exactly, and only, what the instructions say, even if that produces something absurd. Programming is the skill of writing instructions precise enough that there is no gap left to fill.

Why this matters for competitive programming

Almost every "my code is wrong" moment in this book traces back to this one idea: the computer did exactly what you wrote, not what you meant. Learning to read your own code as literally as the computer does is the single most useful debugging habit you can build.

1.2 What is a programming language?

A programming language is a formal language — a fixed set of words and grammar rules — that translates human intent into instructions a computer can execute. C++ is one such language. Its "words" are keywords like `int`, `if`, `return`, and its "grammar" is called syntax — the exact rules for how those words may be combined.

1.3 Compiler vs. interpreter

C++ is a compiled language. A compiler is a separate program that reads your entire C++ source file, checks it for grammar mistakes, and translates it — all at once, before anything runs — into a binary

executable file made of machine instructions the CPU can run directly. This is different from an interpreted language like Python, where another program reads and executes your code line by line, translating as it goes.

	Compiled (C++)	Interpreted (Python)
When is it translated?	Once, before running	Line by line, while running
Typical speed	Very fast	Slower
Errors caught	Many caught before running	Often only when that line runs
Why CP prefers it	Speed matters under a time limit	—

This is exactly why competitive programming is dominated by C++: judges enforce strict time limits (often 1-2 seconds), and a compiled language routinely runs 10-100× faster than an interpreted one for the same algorithm.

1.4 Source code, compilation, and execution

- 1 Source code — the human-readable `.cpp` file you type.
- 2 Compilation — the compiler (e.g. `g++`) reads the source code, checks syntax, and produces a machine-code executable.
- 3 Execution — the operating system runs that executable; your program's instructions actually happen (reading input, computing, printing output).

```
g++ -O2 -std=c++17 solution.cpp -o solution
./solution
```

A typical competitive-programming compile-and-run command on Linux/macOS.

1.5 Syntax, and three kinds of errors

Syntax is the grammar of the language — the exact rules for what is and isn't a legal C++ statement. Break a syntax rule and the compiler refuses to produce an executable at all.

Error type	When it happens	Example
Compilation error	Before the program runs — broken syntax	Missing semicolon; misspelled keyword
Runtime error	While the program is running	Dividing by zero; accessing array index -1
Logical error	Program runs fine but gives the wrong answer	Using < instead of <= in a loop condition

⚠ **Logical errors are the dangerous ones**

Compilation and runtime errors announce themselves. A logical error does not — the program compiles, runs, produces output, and that output is simply wrong. Most of your competitive-programming debugging time will go here, which is why dry-running your code by hand (Section 1.6) is a core skill, not an optional extra.

1.6 Debugging as a mindset

Debugging is not "staring at code until the answer appears." It is a systematic process: form a hypothesis about where the bug is, check it (often by printing intermediate values or tracing by hand on paper), and narrow down the location. Chapter 41 (in a later Part) covers this in depth; for now, the one habit worth building immediately is: when output is wrong, trace your code by hand on the smallest input that reproduces the bug, line by line, writing down the value of every variable as it changes. This is called a dry run, and you will do dozens of them in this book.

Chapter 2 — Your First C++ Program

2.1 The minimal program

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

A complete, runnable C++ program.

Line by line:

- `#include <bits/stdc++.h>` — pulls in (almost) the entire C++ standard library in one line, so every container and algorithm used in this book is available.
- `using namespace std;` — lets you write `cout` instead of `std::cout` everywhere. Saves typing in contest code.
- `int main() { ... }` — every C++ program has exactly one `main` function; execution starts at its first line and ends when it returns or falls off the end.
- `cout << "Hello, World!" << endl;` — prints text to the screen, then moves to a new line.
- `return 0;` — tells the operating system the program finished successfully.

2.2 Why `<bits/stdc++.h>`, really?

In normal software engineering you are taught to include only the specific headers you need (`<vector>`, `<string>`, `<algorithm>`, and so on), because it keeps compile times down and makes dependencies explicit. Competitive programmers break this rule deliberately: in a

contest you might use ten different containers in one file, and re-typing ten include lines under time pressure is wasted effort with zero payoff — nobody but you and the judge ever reads this file. `<bits/stdc++.h>` is a GCC-specific umbrella header that includes essentially everything, trading a slightly longer compile time for zero friction while writing.

▣ Individual headers you should still recognize

For everyday (non-contest) C++ development, or on compilers that don't support `<bits/stdc++.h>` (e.g. clang on macOS by default), you include only what you use: `<iostream>` for I/O, `<vector>`, `<string>`, `<algorithm>`, `<map>`, `<set>`, `<queue>`, etc. Both styles are shown at least once in this book.

2.3 Comments

```
// This is a single-line comment

/* This is a
   multi-line comment */

int x = 5; // comments can also follow code on the same line
```

Comments are ignored entirely by the compiler. Use them to explain why, not what — the code already says what it does; a good comment says why you chose to do it that way.

Chapter 3 — Variables, Constants, and Data Types

3.1 What is a variable?

In one sentence: a variable is a named box in memory that holds a value you can read or change.

When you write `int age = 20;`, the computer reserves a small chunk of memory big enough to hold an integer, labels that chunk `age` so your code can refer to it by name, and stores the value 20 in it. Later, `age = 21;` overwrites the value in that same box — the box does not move, only its contents change.

3.2 Identifiers and naming rules

- May contain letters, digits, and underscores; cannot start with a digit.
- Case-sensitive: `score` and `Score` are different variables.
- Cannot be a reserved keyword (`int`, `return`, etc.).
- Convention in this book: `snake_case` for variables (`max_value`), short names like `i`, `j`, `n` for loop counters and sizes — this matches real competitive-programming style, where code is read once and must be written fast.

3.3 Core data types

C++ is statically typed: every variable's type is fixed at the point you declare it, and the compiler uses that type to know how much memory to reserve and what operations are legal.

Type	Typical size	Typical range	Use for
int	4 bytes	$\approx -2.1 \times 10^9$ to 2.1×10^9	Most whole numbers
long long	8 bytes	$\approx -9.2 \times 10^{18}$ to 9.2×10^{18}	Whole numbers that might overflow int
float	4 bytes	≈ 7 significant digits	Rarely used in CP (low precision)
double	8 bytes	≈ 15 – 16 significant digits	Decimal / real numbers
char	1 byte	single character	Single characters, small integers
bool	1 byte	true / false	Logical flags
string	variable	sequence of characters	Text (from <string>, not built-in)

3.4 Why competitive programmers default to long long

This is one of the highest-value habits in this entire book. An `int` can hold roughly up to 2.1 billion. Many competitive problems have constraints like $n \leq 10^5$ and ask you to sum n values up to 10^9 each — that sum can reach 10^{14} , which silently overflows an `int` and wraps around to a garbage (often negative) value, with no warning from the compiler or a crash at runtime. This single bug — integer overflow — is one of the most common wrong-answer causes on every judge.

```
int n = 100000;
int a[100000]; // imagine every value is 100000
int sum = 0;
for (int i = 0; i < n; i++) sum += a[i]; // sum overflows int
silently!

// Fix: use long long for anything that might exceed  $\sim 2 \times 10^9$ 
long long sum2 = 0;
for (int i = 0; i < n; i++) sum2 += a[i];
```

⚠ Habit to build now

Whenever you compute a sum, product, or count that could plausibly exceed a few hundred million, use `long long` for it — even if the individual inputs fit in `int`. Many experienced competitive programmers write `#define int long long` at the top of their template for exactly this reason (with signed `main()` instead of `int main()`).

3.5 Signed vs. unsigned, and overflow behaviour

`int` is signed by default (can be negative). `unsigned int` uses the same 4 bytes but only represents non-negative numbers, roughly doubling the positive range. Mixing signed and unsigned values in one expression is a classic source of subtle bugs, because C++ silently converts the signed value to unsigned, and a negative number becomes an enormous positive one.

```
int a = -1;
unsigned int b = 1;
if (a < b) cout << "a is smaller\n";
else cout << "a is NOT smaller\n"; // this prints! -1 is converted
to a huge unsigned value
```

⚠ Common mistake

Comparing `int` to `size()` of an STL container (which returns an unsigned type) is a frequent source of this exact bug. Prefer comparing with the same signedness, or cast explicitly.

3.6 Type conversion

Implicit conversion happens automatically when C++ needs a different type than the one you gave it. Explicit conversion (a cast) is when you request the conversion yourself.

```
int a = 7, b = 2;
cout << a / b << endl; // 3 – integer division truncates
cout << (double)a / b << endl; // 3.5 – explicit cast makes it a real
division
double d = a; // 7.0 – implicit conversion, int to double
```

Dry run

Line	Expression	Result
1	<code>a / b</code>	<code>7 / 2 = 3</code> (integer division discards the remainder)
2	<code>(double)a / b</code>	<code>a</code> is cast to <code>7.0</code> first, so <code>7.0 / 2 = 3.5</code>
3	<code>double d = a</code>	<code>a</code> 's value <code>7</code> is widened to <code>7.0</code>

⚠ The #1 beginner bug in competitive math problems

`int` divided by `int` is always `int` division in C++ — the fractional part is thrown away, not rounded. If you need a real-number result, cast at least one operand to `double` before the division happens.

3.7 Constants

```
const int MOD = 1e9 + 7;  
const double PI = 3.14159265358979;
```

`const` tells the compiler this value must never change after initialization; any attempt to reassign it is a compile error. It is used constantly in competitive code for fixed values like a modulus.

Chapter 3 quick revision

Variables are named memory boxes; their type is fixed at declaration.

Default to `long long` whenever a sum/count/product might exceed $\sim 2 \times 10^9$.

Integer $\div$ integer truncates in C++ — cast to double for real division.

Avoid mixing signed and unsigned types in comparisons.

Practice — Chapter 3

Problem 3.1 — Predict the output [Level 0]

What does the following print?

```
int a = 9, b = 4; cout << a / b << " " << (double)a / b;
```

Hint 1: Remember: `int / int` truncates.

Approach: `a/b` performs integer division: $9/4 = 2$ (remainder discarded). Casting `a` to double before dividing gives $9.0/4 = 2.25$.
Output: 2 2.25.

Problem 3.2 — Spot the overflow [Level 1]

A problem has $n \leq 10^5$ and each value up to 10^9 . You need the sum of all n values. Which type should the sum variable be, and why?

Key observation: Worst case sum $\approx 10^5 \times 10^9 = 10^{14}$, which exceeds `int`'s $\sim 2.1 \times 10^9$ limit.

Approach: Use `long long` for the sum (the individual array elements can stay `int` if each fits, but the accumulator must not).

Chapter 4 — Input and Output for Competitive Programming

4.1 Basic `cin` and `cout`

```
int a, b;  
cin >> a >> b;  
cout << a + b << endl;
```

`cin >>` automatically skips leading whitespace (spaces, tabs, newlines) and stops reading a value at the next whitespace. Chaining `cin >> a >> b` reads two whitespace-separated values in one line — you almost never need to worry about exactly how many physical lines the input spans.

4.2 Reading lines with `getline`

`cin >>` stops at whitespace, so it cannot read a line containing spaces (like a full sentence). `getline(cin, s)` reads an entire line, including spaces, into a string, up to the next newline.

```
string name;  
getline(cin, name); // reads "John Smith" as one string, not just  
"John"
```

⚠ The classic `cin >> / getline` trap

If you do `cin >> n;` and then `getline(cin, s);` right after, the `getline` will read an empty string. Why? `cin >> n` leaves the newline character after the number still sitting in the input buffer, and `getline` immediately reads up to that leftover newline — getting nothing. Fix by consuming it first: `cin.ignore();` before the `getline` call.

4.3 Fast I/O

By default, C++'s `cin/cout` are synchronized with C's old-style `scanf/printf` so that mixing the two styles in one program still works correctly. That synchronization has a real performance cost, and when a problem needs to read or print hundreds of thousands of values, it can be the difference between passing and getting a Time Limit Exceeded verdict.

```
ios::sync_with_stdio(false);  
cin.tie(nullptr);
```

Put these two lines at the very top of `main()`. `sync_with_stdio(false)` disables the synchronization with C's I/O (safe as long as you don't mix `scanf/printf` with `cin/cout` in the same program). `cin.tie(nullptr)` stops `cin` from automatically flushing `cout` before every read, which also has a cost when reading a lot of input.

📌 Practical rule

Add these two lines to every competitive-programming template, unconditionally. There is essentially never a downside in a contest setting.

4.4 Common competitive input formats

Recognizing the input format instantly, without re-reading it twice, saves real time in a contest. The four you will see constantly:

Format A — T test cases

```
int t;
cin >> t;
while (t--) {
    // read and solve one test case here
}
```

Format B — N followed by N numbers

```
int n;
cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];
```

Format C — N and M, then an N×M grid

```
int n, m;
cin >> n >> m;
vector<vector<int>> grid(n, vector<int>(m));
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cin >> grid[i][j];
```

Format D — Multiple test cases with varying sizes

```
int t;
cin >> t;
while (t--) {
    int n;
    cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x; // range-based loop, explained in
    Chapter 7
    // solve this test case
}
```

Chapter 4 quick revision

`cin >>` skips whitespace and stops at the next whitespace; `getline` reads a full line including spaces.

Add `ios::sync_with_stdio(false); cin.tie(nullptr);` to every contest template.

Learn to recognize the T-test-cases / N-then-N-numbers / N-M-grid input shapes on sight.

Practice — Chapter 4

Problem 4.1 — Sum per test case [Level 0]

Read an integer T , then for each of T test cases read an integer n followed by n integers, and print their sum.

Hint 1: Use the 'T test cases' pattern (Format A) nested with the 'N then N numbers' pattern (Format B).

Complexity: $O(\text{sum of all } n)$ time, well within limits for typical constraints.

Chapter 5 — Operators

5.1 Arithmetic and modulo

Operator	Meaning	Example (a=7, b=2)
+	addition	9
-	subtraction	5
*	multiplication	14
/	division (int truncates)	3
%	modulo (remainder)	1

% (modulo) gives the remainder after integer division and is everywhere in competitive programming: checking even/odd ($n \% 2$), wrapping an index around a circular structure, and computing answers modulo a large prime (Chapter 62, later Part).

⚠ Negative modulo in C++

In C++, the result of % can be negative if the left operand is negative: $-7 \% 3$ is -1 , not 2 as in some other languages. To force a non-negative result: $((a \% m) + m) \% m$.

5.2 Relational and logical operators

Operator	Meaning
==	equal to
!=	not equal to
<, >, <=, >=	comparisons
&&	logical AND
	logical OR
!	logical NOT

⚠️ == vs =

= assigns a value; == compares two values. Writing `if (x = 5)` by accident is legal C++ (it assigns 5 to x, and the condition becomes "is 5 truthy", which is always true) — a classic, hard-to-spot bug.

&& and || short-circuit: in `a() && b()`, if `a()` is false, `b()` is never even evaluated (since the whole expression must be false regardless). This is used deliberately to guard against errors:

```
if (i < (int)v.size() && v[i] == target) { /* safe: size check happens first */ }
```

5.3 Increment/decrement, compound assignment, ternary

```
int x = 5;
x++; // x becomes 6 (post-increment: use old value, then increment)
++x; // x becomes 7 (pre-increment: increment, then use new value)
x += 3; // x = x + 3
int y = (x > 5) ? 100 : 200; // ternary: y = 100 if x > 5, else 200
```

□ Pre vs post increment

For a simple standalone statement like `i++`; in a loop, pre and post increment behave identically in effect. The difference only matters when the increment's result is used in the same expression, e.g. `a[i++]` vs `a[++i]`.

5.4 Bitwise operators

Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise XOR
~	bitwise NOT
<<	left shift (multiply by 2 per shift)
>>	right shift (divide by 2 per shift)

These operate on the binary representation of a number, bit by bit. A full chapter is dedicated to bit manipulation later (Chapter 16); for now, know that `1 << k` is a fast way to compute 2 raised to the power `k`, and is used constantly for representing subsets as bitmasks.

5.5 Operator precedence and associativity

When an expression mixes operators, precedence decides which runs first — exactly like "multiplication before addition" in arithmetic. When two operators have equal precedence, associativity decides the order (left-to-right for most operators).

```
int r = 2 + 3 * 4; // 14, not 20 — * binds tighter than +
int s = 10 - 3 - 2; // 5 — subtraction is left-associative: (10-3)-2
bool t = 1 < 2 && 3 < 4; // true — < binds tighter than &&
```

□ When in doubt, parenthesize

You do not need to memorize the full precedence table. Professional and competitive programmers alike add parentheses whenever an expression's order of evaluation isn't instantly obvious — it costs nothing and removes an entire category of bugs.

Practice — Chapter 5 (output prediction)

Problem 5.1 — Predict the output [Level 0]

`int a = 5; cout << (a++ + ++a);` — what is printed, and what is `a` afterward?

Hint 1: `a++` uses 5, then `a` becomes 6. `++a` then makes `a` become 7 and uses 7.

Approach: $5 + 7 = 12$ is printed. `a` ends up as 7. (Mixing multiple increments of the same variable in one expression is technically undefined behaviour before evaluation order was tightened in C++17 for some cases — in real contest code, avoid writing expressions like this at all.)

Problem 5.2 — Negative modulo [Level 1]

Write an expression that computes `a % m` for possibly-negative `a`, guaranteeing a result in $[0, m)$.

Approach: `((a % m) + m) % m`

Chapter 6 — Conditions

6.1 if, else if, else

```
int n;
cin >> n;
if (n > 0) {
    cout << "positive";
} else if (n < 0) {
    cout << "negative";
} else {
    cout << "zero";
}
```

Conditions are checked top to bottom; the first one that is true runs, and the rest are skipped entirely — even if a later condition would also be true.

6.2 Nested conditions

```
if (n % 4 == 0) {  
    if (n % 100 != 0 || n % 400 == 0) cout << "leap year";  
    else cout << "not leap year";  
} else {  
    cout << "not leap year";  
}
```

A year is a leap year if divisible by 4, unless it's also divisible by 100 — unless it's also divisible by 400. This is a classic example where nested conditions map directly onto a nested real-world rule.

6.3 switch and the ternary operator

```
switch (grade) {  
    case 'A': cout << "Excellent"; break;  
    case 'B': cout << "Good"; break;  
    default: cout << "Keep trying"; break;  
}
```

⚠ Don't forget break

Without `break`, execution "falls through" into the next case — sometimes intentional, usually a bug.

The ternary operator (Section 5.3) is a compact if/else that produces a value rather than running a block: use it for a simple choice between two expressions, and a full if/else when either branch needs more than one statement.

Practice — Chapter 6

Problem 6.1 — FizzBuzz [Level 0]

For a given n , print "Fizz" if divisible by 3, "Buzz" if divisible by 5, "FizzBuzz" if divisible by both, otherwise print n itself.

Hint 1: Check the 'both' condition first, or check divisibility by 15 directly.

Key observation: If you check 'divisible by 3' before 'divisible by both', you must make sure the 'both' case doesn't get swallowed by the 'by 3' branch first — order matters here.

Approach: if ($n \% 15 == 0$) Fizzbuzz; else if ($n \% 3 == 0$) Fizz; else if ($n \% 5 == 0$) Buzz; else print n .

Problem 6.2 — Largest of three [Level 0]

Read three integers and print the largest, without using any library max function.

Approach: Nested if/else comparing pairs, or a chain of ternary operators.

Chapter 7 — Loops

7.1 for, while, do-while

```
// for: best when you know the number of iterations in advance
for (int i = 0; i < n; i++) { /* ... */ }

// while: best when the stopping condition isn't a simple counter
while (x > 1) { x /= 2; }

// do-while: runs the body at least once, checks condition after
int choice;
do { cin >> choice; } while (choice != 0);
```

A for loop has three parts separated by semicolons: initialization (runs once), condition (checked before every iteration), and update (runs after every iteration). All three are optional, but the two semicolons are always required.

7.2 break, continue, and nested loops

```
for (int i = 0; i < n; i++) {  
    if (a[i] == 0) continue; // skip the rest of this iteration, go  
    to next i  
    if (a[i] < 0) break; // exit the loop entirely  
    cout << a[i] << " ";  
}
```

⚠ break only exits the innermost loop

In nested loops, break exits only the loop it's directly written inside. To exit multiple nested loops at once, use a flag variable, restructure into a function and return, or (rarely, and controversially) a goto.

```
bool found = false;  
for (int i = 0; i < n && !found; i++) {  
    for (int j = 0; j < m; j++) {  
        if (grid[i][j] == target) { found = true; break; }  
    }  
}
```

7.3 Common loop patterns

Certain shapes come up so often that recognizing them instantly is worth practicing directly:

Digit extraction

```
int n = 4527, sumDigits = 0;
while (n > 0) {
    sumDigits += n % 10; // last digit
    n /= 10; // remove last digit
}
```

Dry run

n before	n % 10	sumDigits	n after
4527	7	7	452
452	2	9	45
45	5	14	4
4	4	18	0

Palindrome check (number)

```
int original = n, reversed = 0;
while (n > 0) {
    reversed = reversed * 10 + n % 10;
    n /= 10;
}
bool isPalindrome = (reversed == original);
```

7.4 Complexity of loops (a first look)

A single loop that runs n times does $O(n)$ work — the running time grows linearly with n . A loop nested inside another loop, each running n times, does $O(n \times n) = O(n^2)$ work, because the inner loop's full n iterations happen for every single iteration of the outer loop. This idea — multiplying the iteration counts of nested loops — is the seed of everything in Chapter 17 (Complexity), a full Part of its own later in this book.

```
for (int i = 0; i < n; i++) // runs n times
    for (int j = 0; j < n; j++) // runs n times, for each i
        cout << i * j; // O(n * n) = O(n^2) total print operations
```

❏ Why this matters immediately

With $n \leq 10^5$, an $O(n^2)$ loop performs about 10^{10} operations — far too slow for a typical 1-second time limit (roughly 10^8 – 10^9 simple operations per second). Recognizing this before you finish writing the code, just from the constraints, saves enormous amounts of wasted implementation time. This is exactly the constraints-to-complexity table from the Master Roadmap.

📌 Chapter 7 quick revision

Use `for` when the iteration count is known in advance, while otherwise.

`break` exits only the innermost loop; use a flag or a function return to escape nested loops.

Nested loops multiply their iteration counts for complexity purposes: two nested n -loops $\Rightarrow O(n^2)$.

Practice — Chapter 7

Problem 7.1 — Reverse a number [Level 0]

Given an integer n , print its digits in reverse order (e.g. $1230 \rightarrow 321$).

Hint 1: Use the digit-extraction pattern ($n \% 10$, $n /= 10$) shown above.

Approach: Build the reversed number digit by digit: $\text{reversed} = \text{reversed} * 10 + n \% 10$, then $n /= 10$, repeated until n becomes 0.

Problem 7.2 — Count pairs with a given sum [Level 2]

Given an array of n integers and a target sum S , count how many pairs (i, j) with $i < j$ satisfy $a[i] + a[j] == S$, using the most direct approach.

Hint 1: Try every pair with two nested loops first — this is the brute force.

Key observation: This brute force is $O(n^2)$, which is fine for n up to a few thousand but too slow for $n = 10^5$. A faster $O(n)$ hash-map approach is covered once hashing is introduced.

Complexity: $O(n^2)$ time, $O(1)$ extra space.

Chapter 8 — Functions

8.1 Why functions?

In one sentence: a function is a named, reusable block of code that takes inputs (parameters) and optionally produces an output (a return value).

Functions let you write a piece of logic once and use it many times, give a complex operation a meaningful name, and break a large problem into small, testable pieces — all essential once problems get bigger than a few lines.

8.2 Declaration, definition, parameters, return

```
// declaration + definition combined
int square(int x) { // x is a parameter
    return x * x;
}

int main() {
    int result = square(5); // 5 is the argument passed in
    cout << result; // 25
}
```

Parameters are the named placeholders in the function's definition; arguments are the actual values you pass when calling it. A function that returns nothing uses `void` as its return type and has no return value; statement (a bare `return`; is still allowed, to exit early).

8.3 Pass by value vs. pass by reference

By default, C++ passes arguments by value: the function receives a copy of the argument, and changes inside the function do not affect the caller's variable. Pass by reference (using `&`) instead gives the function direct access to the original variable.

```
void addOneByValue(int x) { x += 1; } // no effect outside the
function
void addOneByRef(int &x) { x += 1; } // modifies the caller's
variable

int main() {
    int a = 5;
    addOneByValue(a);
    cout << a << " "; // 5 – unchanged
    addOneByRef(a);
    cout << a; // 6 – changed
}
```

❏ Why references matter in competitive programming

Pass-by-reference is used for two reasons: (1) you want the function to modify the caller's data (e.g. sorting an array in place), or (2) performance — passing a large vector or string by value copies the entire thing, which is slow. When you don't want the function to modify the argument but still want to avoid the copy, pass by const reference: `void f(const vector<int> &v).`

8.4 Default arguments and overloading

```
void greet(string name, string greeting = "Hello") {  
    cout << greeting << ", " << name << "!";  
}  
greet("Ana"); // Hello, Ana!  
greet("Ana", "Hi"); // Hi, Ana!  
  
// Overloading: same name, different parameter types/counts  
int add(int a, int b) { return a + b; }  
double add(double a, double b) { return a + b; }
```

8.5 Scope: local vs. global variables

A variable declared inside a function (or any `{ }` block) is local: it exists only within that block and is destroyed when the block ends. A variable declared outside any function is global: it exists for the entire program's run and is visible from every function.

⚠ Global state is convenient but dangerous

Competitive programmers often use global arrays for graphs/DP tables specifically because it avoids passing large structures as parameters repeatedly. In larger software, uncontrolled global state is a common source of hard-to-trace bugs — know both the convenience and the risk.

8.6 A first look at recursion

A function that calls itself is recursive. Chapter 15 (in a later Part) covers recursion in depth, including the call stack and backtracking; here is the shape, so it isn't unfamiliar when functions with self-calls appear earlier in examples:

```
int factorial(int n) {  
    if (n <= 1) return 1; // base case – stops the recursion  
    return n * factorial(n - 1); // recursive case – calls itself  
    with a smaller input  
}
```

8.7 Lambda functions (a preview)

A lambda is a small, unnamed function you can write inline, most often used as a custom comparator for sorting (Chapter 18 covers this in full):

```
auto isEven = [](int x) { return x % 2 == 0; };  
cout << isEven(4); // 1 (true)  
  
sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); //  
descending sort
```

Chapter 8 quick revision

Pass by value copies; pass by reference (&) lets a function modify the caller's variable.

Use `const &` to avoid copying large containers without allowing modification.

Local variables live only inside their block; global variables live for the whole program.

Every recursive function needs a base case that stops the recursion.

Practice — Chapter 8

Problem 8.1 — Swap by reference [Level 0]

Write a function `void mySwap(int &a;, int &b;)` that swaps two integers without returning anything.

Approach: `int temp = a; a = b; b = temp;` — works because `a` and `b` are references to the caller's variables, so modifying them here modifies the originals.

Problem 8.2 — Why does this not swap? [Level 1]

A student writes `void mySwap(int a, int b) { int t=a; a=b; b=t; }` and is confused why calling it doesn't swap their variables. Explain the bug.

Key observation: The parameters are pass-by-value, so `a` and `b` inside the function are copies; swapping the copies has no effect on the caller's original variables.

Approach: Change the signature to take references: `int &a;, int &b;.`

Chapter 9 — Pointers and References

9.1 Memory and addresses

Every variable lives somewhere in the computer's memory, and that location has a numeric address. A pointer is a variable whose value is an address — it "points to" the location of another variable, rather than holding a value directly.

```
int x = 10;
int *p = &x; // p holds the address of x
cout << x; // 10 – the value
cout << &x; // e.g. 0x7ffee3... – the address
cout << p; // same address as &x
cout << *p; // 10 – dereferencing: "the value at the address p
holds"
```

& here means address-of (a different meaning from reference-declaration — context tells them apart). * here means dereference — "go to this address and give me the value stored there".

9.2 Pointers and functions, pointer arithmetic, nullptr

```
void increment(int *p) { (*p)++; } // modifies the original via its
address

int arr[3] = {10, 20, 30};
int *pa = arr; // arrays decay into a pointer to their first element
cout << *(pa + 1); // 20 – pointer arithmetic: move forward 1
int-width

int *empty_ptr = nullptr; // points to nothing; safe, explicit "no
address" value
```

⚠ Dangling pointers

A pointer that still holds an address after the memory at that address has been freed or gone out of scope is called dangling. Dereferencing it is undefined behaviour — it might crash, or it might silently return garbage. Always set a pointer to `nullptr` once what it points to is no longer valid, and check for `nullptr` before dereferencing.

9.3 References

A reference is an alias — another name for an existing variable, not a separate object. Unlike a pointer, a reference must be initialized when

declared and can never be reassigned to refer to something else afterward.

	Pointer	Reference
Can be null / uninitialized?	Yes (nullptr)	No — must bind to a real variable
Can be reassigned to something else?	Yes	No, once bound
Needs dereferencing to use the value?	Yes (*p)	No — used just like the original variable
Typical use	Optional/nullable data, dynamic memory, arrays	Function parameters (avoid copies / allow modification)

This is exactly why Section 8.3 used `int &x` for pass-by-reference: references give the safety of "must refer to something real" while still letting a function reach into and modify the caller's variable.

Chapter 9 quick revision

A pointer stores an address; `*` dereferences it to get the value there.

A reference is an alias for an existing variable — no separate address arithmetic, no re-binding.

Never dereference a null or dangling pointer.

In modern competitive C++, references are used constantly (function parameters, range-based loops); raw pointers are used sparingly, mostly for tree/linked-list nodes (a later Part).

Practice — Chapter 9

Problem 9.1 — Predict the output [Level 1]

```
int a = 3; int *p = &a; *p = 10; cout << a; — what prints?
```

Approach: 10 — `*p = 10` writes through the pointer into `a`'s own memory location, since `p` holds `a`'s address.

Problem 9.2 — Reference vs pointer rewrite [Level 1]

Rewrite `void inc(int *p) { (*p)++; }`, called as `inc(&x;)`, to use a reference instead of a pointer.

Approach: `void inc(int &p;) { p++; }`, called simply as `inc(x)` — no address-of needed at the call site.

Part I — Master Revision

Must Remember

- The computer does exactly what your code says, not what you meant — read code literally when debugging.
- Default to `long long` for any sum/count/product that could exceed $\sim 2 \times 10^9$.
- `int / int` truncates; cast to double before dividing if you need a real-number result.
- Add `ios::sync_with_stdio(false); cin.tie(nullptr);` to every contest template.
- Nested loops multiply iteration counts: two nested `n`-loops give $O(n^2)$ — check this against constraints before you even finish writing the code.
- Pass by reference (or const reference) to modify a caller's variable or avoid copying large data.
- A pointer holds an address and needs `*` to dereference; a reference is a direct alias with no dereferencing needed.

Common Mistakes Recap

Mistake	Fix
Using int for a sum that can exceed 2×10^9	Use long long
int/int division when a real result is needed	Cast to double before dividing
Writing if (x = 5) instead of if (x == 5)	Always double-check == in conditions
getline reading an empty line right after cin >>	Call cin.ignore() first
Comparing signed int to unsigned .size()	Cast explicitly or compare same signedness
Forgetting break in a switch statement	Add break after every case unless fall-through is intended
Passing a large vector/string by value	Pass by const reference
Dereferencing a null or dangling pointer	Check for nullptr; null out pointers after use

Complexity Table So Far

Code shape	Complexity
Single loop over n elements	$O(n)$
Two nested loops, both over n	$O(n^2)$
Loop that halves a value each time (e.g. $n /= 2$)	$O(\log n)$
Fixed number of operations regardless of n	$O(1)$

Interview-Style Questions

- 1 Why do competitive programmers prefer long long over int for most quantities?

- 2 What is the practical difference between pass-by-value and pass-by-reference?
- 3 Why does `ios::sync_with_stdio(false)` speed up I/O, and when would you avoid using it?
- 4 What is the difference between a compilation error, a runtime error, and a logical error?
- 5 Why can a reference not be reseated to refer to a different variable after initialization?

Cheat Sheet — Part I

```
// Standard competitive-programming template so far
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        vector<int> a(n);
        for (auto &x : a) cin >> x;
        long long sum = 0;
        for (int x : a) sum += x;
        cout << sum << "\n";
    }
    return 0;
}
```

□ Coming up in Part II

Part II — Problem-Solving Fundamentals builds the thinking process that sits on top of this syntax: how to read a problem statement, extract constraints, find a brute force, and reason about when it's fast enough — the habits that actually separate someone who knows C++ from someone who can solve unfamiliar problems with it.